Advanced Causal Inference

Lecture 08: Heterogeneous Treatment Effects & Optimal Treatment Regime

Chan Park
University of Illinois, Urbana-Champaign

This lecture note is partially based on the lecture notes/textbook of Wager (2024), Kosorok and Moodie (2015)

Outline

1. Heterogeneous Treatment Effect
2. (Single Time-point) Optimal Treatment Regime
3. Assignment

Review

Assumption (SUTVA (Rubin, 1980))

- $Y = Y(A) = AY(1) + (1 - A)Y(0)$

Assumption (Conditional Ignorability)

- $Y(a) \perp\!\!\!\perp A \,|\, X$, $\forall a$

Assumption (Positivity/Overlap)

- $\Pr(A = a \,|\, X) \in (0, 1)$, $\forall (a, X)$

- Under these assumptions,

$$
\begin{aligned}
\tau_{\mathsf{ATE}} &= \mathrm{E}\{Y(1) - Y(0)\} \\
&= \mathrm{E}\big\{\mathrm{E}(Y|A = 1, X) - \mathrm{E}(Y|A = 0, X)\big\} && \text{OR-based Representation} \\
&= \mathrm{E}\left\{ \frac{AY}{\Pr(A = 1|X)} - \frac{(1 - A)Y}{\Pr(A = 0|X)} \right\} && \text{IPW-based Representation}
\end{aligned}
$$

Conditional Average Treatment Effect

- We also have the following identification result for the **conditional average treatment effect** (CATE):

$$\tau_{\text{CATE}}(x) = \mathrm{E}\{Y(1) - Y(0)|X = x\} = \mathrm{E}(Y|A = 1, X = x) - \mathrm{E}(Y|A = 0, X = x) \in \mathcal{F}$$

  where $d = \dim(X)$ and $\mathcal{F} = \{f|f : \mathbb{R}^d \to \mathbb{R}\}$

- This is the average treatment effect among individuals with characteristics $X = x$

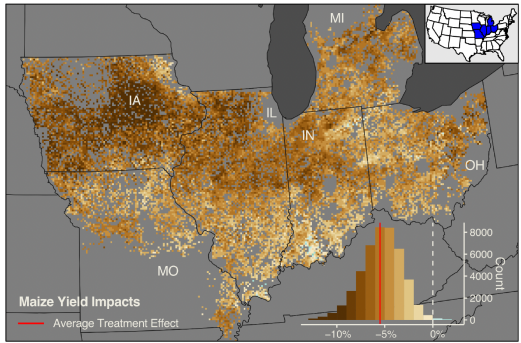
- The CATE can be used to study treatment effect heterogeneity and to inform decision-making about whether or not to treat a given individual with X
  - If X is associated with a positive CATE, individuals with X benefit more from the treatment on average
  - If X is associated with a negative CATE, individuals with X be harmed by the treatment on average

- Remarks:
  - This is different from the individual treatment effect $Y_i(1) - Y_i(0)$, which is not identifiable under the assumptions
  - τ_{CATE} is generally an infinite-dimensional parameter, whereas τ_{ATE} is a scalar (one-dimensional) parameter
  - Thus, $\widehat{\tau}_{\text{CATE}}$ is generally not $\sqrt{N}$-estimable, whereas τ_{ATE} may be

Example: CATE

- Deines et al. (2023)
  - Treatment: cover cropping (the practice of planting non-commercial crops between regular growing seasons)
  - Outcome: maize yield
  - Covariates: precipitation, temperature, sun light, etc

- ATE alone isn't enough for informed decisions



T- and S-Learner

- Let $\mu(a, X) = \mathrm{E}(Y|A = a, X)$
- T- and S-Learner (Künzel et al., 2019):

Algorithm 1: T-learner (T=Two)

Estimate $\mu(a, X)$ for $a = 0, 1$ from $\texttt{lm}(Y \sim X \mid A = a)$ or $\texttt{ML}(Y \sim X \mid A = a)$;
return $\widehat{\tau}_T(X) = \widehat{\mu}(1, X) - \widehat{\mu}(0, X)$

- Performs well if there are no common trends in $\mu(0, X)$ and $\mu(1, X)$ and if the CATE is very complicated
- Because data is not pooled across treatment groups, it is difficult for the T-learner to mimic a behavior (e.g., discontinuity) that appears in all the treatment groups

Algorithm 2: S-learner (S=Single)

Estimate $\mu(A, X)$ from $\texttt{lm}(Y \sim (A, X))$ or $\texttt{ML}(Y \sim (A, X))$; interaction terms $A : X$ can be included;
return $\widehat{\tau}_S(X) = \widehat{\mu}(1, X) - \widehat{\mu}(0, X)$

- Since A plays no special role, the base learners can completely ignore it during model-fitting; this is good if the CATE is zero in many places, i.e., the S-learner can be biased toward zero
- For some base learners (e.g., k-nearest neighbors), treating A like X may not make sense

Regularization Bias

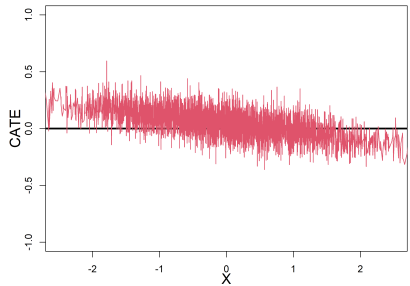
- While the T-learner is consistent, it may not perform well in finite samples due to "regularization bias"
- Since $\hat{\mu}(1, X)$ and $\hat{\mu}(0, X)$ are fitted separately, these fts may end up being regularized in different ways
- This creates bias in $\hat{\tau}_T(X)$
- Example

$$X \sim N_{100}(0, I_{100}) \ , \qquad \text{logit}\{\Pr(A = 1|X)\} = 0.5X_1 - 1$$

$$\mu(a, X) = \sum_{j=1}^{10} X_j \ , \qquad \tau_{\text{CATE}}(X) = 0$$

$$Y(a) = \mu(a, X) + \epsilon \ , \qquad \epsilon \sim N(0, 1)$$

  - Black: True CATE; Red: Estimated CATE



- $\hat{\tau}_T$ shows greater variability than the true CATE

Regularization Bias

```
expit <- function(v){exp(v)/(1+exp(v))}
N <- 5000
X <- matrix(rnorm(N*100),N,100)
X <- X[order(X[,1]),]
A <- rbinom(N,1,expit(0.5*X[,1]-1))
mu1 <- mu0 <- X%*%c(rep(1,10),rep(0,90))
Y1 <- mu1 + rnorm(N); Y0 <- mu0 + rnorm(N); Y <- A*Y1+(1-A)*Y0

library(glmnet)

mu1.Lasso <- cv.glmnet(x=X[A==1,], y=Y[A==1]); mu0.Lasso <- cv.glmnet(x=X[A==0,], y=Y[A==0])

beta1 <- as.vector(coef(mu1.Lasso, s = "lambda.1se"))
beta0 <- as.vector(coef(mu0.Lasso, s = "lambda.1se"))

CATE.Est <- cbind(1,X)%*%(beta1-beta0); CATE.True <- mu1 - mu0

plot(X[,1],CATE.True,type='l',xlim=c(-2.5,2.5),ylim=c(-1,1),xlab="",ylab="",lwd=3)
par(new=TRUE)
points(X[,1],CATE.Est,type='l',pch=19,cex=0.25,col=2,xlab="",ylab="",lwd=1.25)
title(xlab="X",ylab="CATE",cex.lab=1.5,line=2)
```

Regularization Bias

- $\widehat{\tau}_T$ shows greater variability than the true CATE
- This issue becomes more pronounced:
  - when the regularization depends on the sample size;
  - when the treated and control groups are highly imbalanced
- Also, T-learner does not explicitly account for variation in the propensity score
  - If $e(X)$ varies considerably, then $\widehat{\mu}(a, X)$ will be driven by data in $\mathcal{X}_0 = \{X|e(X) \simeq 0\}$ and $\mathcal{X}_1 = \{X|e(X) \simeq 1\}$
  - And if $\mathcal{X}_0$ and $\mathcal{X}_1$ are substantially different (i.e., covariate shift), this may create bias for their difference $\widehat{\tau}_T(X)$

X-Learner

- X-Learner (Künzel et al., 2019):

Algorithm 3: X-Learner (X=Cross; as we mix two learners)

Estimate $\mu(a, X)$ for $a = 0, 1$ from $\texttt{lm}(Y \sim X \mid A = a)$ or $\texttt{ML}(Y \sim X \mid A = a)$;
Define a pseudo-ITE: $\widetilde{\Delta} = Y - \widehat{\mu}(0, X)$ if $A = 1$; $\widetilde{\Delta} = \widehat{\mu}(1, X) - Y$ if $A = 0$;
Estimate the CATE from lm($\widetilde{\Delta} \sim X | A = a$) or ML($\widetilde{\Delta} \sim X | A = a$), denoted by $\widehat{\tau}_a$;
Merge two CATE estimates: $\widehat{\tau}_X(x) = g(x)\widehat{\tau}_1(x) + \{1 - g(x)\}\widehat{\tau}_0(x)$; one can use $g = \widehat{e}$;
return $\widehat{\tau}_X(x)$

- Comparisons to T- and S-learners
  - The X-learner can adapt to structural properties such as sparsity or smoothness of the CATE
  - When the CATE is zero, it usually is not as good as the S-learner but is better than the T-learner
  - When the CATE is complex, it outperforms the S-learner, and is often better than the T-learner
  - It is particularly effective when the treated and control groups are highly imbalanced

- Implemented in `causalToolbox` R package

X-Learner

```
mu1.Lasso <- cv.glmnet(x=X[A==1,], y=Y[A==1])
mu0.Lasso <- cv.glmnet(x=X[A==0,], y=Y[A==0])
beta1 <- as.vector(coef(mu1.Lasso, s = "lambda.1se"))
beta0 <- as.vector(coef(mu0.Lasso, s = "lambda.1se"))
mu1.est <- cbind(1,X)%*%beta1; mu0.est <- cbind(1,X)%*%beta0

e.Lasso <- cv.glmnet(x=X,y=A,family="binomial")
e.gamma <- as.vector(coef(e.Lasso, s = "lambda.1se"))
e.est <- expit(cbind(1,X)%*%e.gamma)

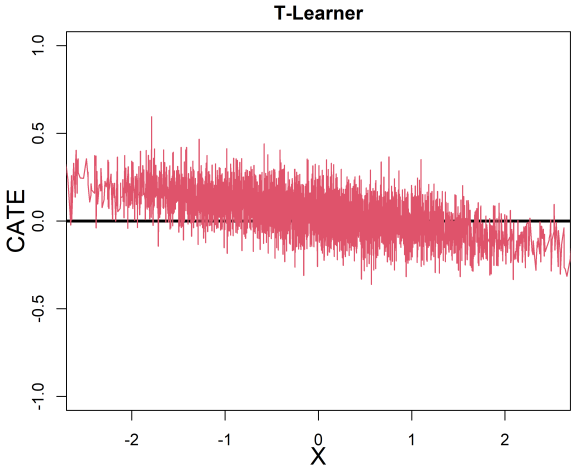
Delta <- A*(Y - mu0.est)+(1-A)*(mu1.est-Y)

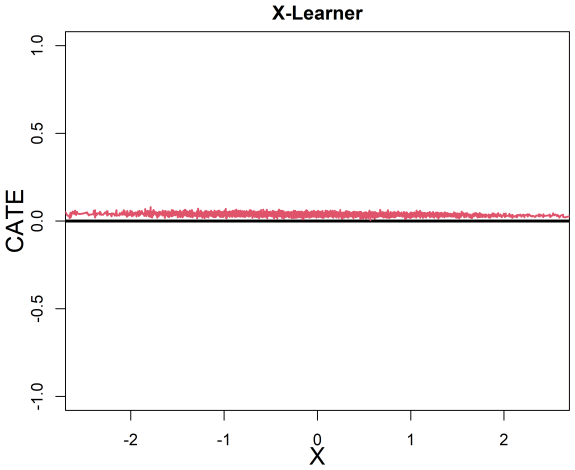
tau1.Lasso <- cv.glmnet(x=X[A==1,], y=Delta[A==1]);
tau0.Lasso <- cv.glmnet(x=X[A==0,], y=Delta[A==0])

X.beta1 <- as.vector(coef(tau1.Lasso, s = "lambda.1se"))
X.beta0 <- as.vector(coef(tau0.Lasso, s = "lambda.1se"))
X.CATE1.est <- cbind(1,X)%*%X.beta1; X.CATE0.est <- cbind(1,X)%*%X.beta0
X.CATE.est <- e.est*X.CATE1.est + (1-e.est)*X.CATE0.est

plot(X[,1],CATE.True,type='l',xlim=c(-2.5,2.5),ylim=c(-1,1),xlab="",ylab="",lwd=3)
par(new=TRUE)
points(X[,1],X.CATE.est,type='l',pch=19,cex=0.25,col=2,xlab="",ylab="",lwd=2)
title(xlab="X",ylab="CATE",cex.lab=1.5,line=2)
title(main="X-Learner")
```

X-Learner


T-Learner
CATE
1.0
0.5
0.0
-0.5
-1.0
-2
-1
0
1
2
X


X-Learner
CATE
1.0
0.5
0.0
-0.5
-1.0
-2
-1
0
1
2
X

PLM-Learner

- We "focus" on estimating the CATE accurately
- This is not necessarily the same thing as simultaneously estimating $\mu(1, X)$ and $\mu(0, X)$ accurately
- One can write $\mathrm{E}(Y|A = a, X) = \mu(0, X) + a\tau_{\mathsf{CATE}}(X)$
- In order to avoid overfitting, one can model

$$\tau_{\mathsf{CATE}}(X) = \beta^{\intercal}\psi(X)$$

  which implies

$$\mathrm{E}(Y|A = a, X) = \mu(0, X) + a\beta^{\intercal}\psi(X)$$

- This model is often referred to as a partially linear model (Robinson, 1988)

PLM-Learner

- Since $m(X) := \mathrm{E}(Y|X) = e(X)\mathrm{E}(Y|A=1,X) + \{1-e(X)\}\mathrm{E}(Y|A=0,X)$, we have

$$Y - m(X) = \{A - e(X)\}\beta^\intercal \psi(X) + \epsilon$$

- Therefore, β can be estimated from the following two-step procedure

Algorithm 4: PLM-Learner

Estimate $m(X)$ and $e(X)$;

Estimate β by regressing $\widetilde{Y} = Y - \widehat{m}(X)$ on $\widetilde{Z} = \{A - \widehat{e}(X)\}\psi(X)$;

return $\widehat{\tau}_{PLM}(X) = \widehat{\beta}^\intercal \psi(X)$

- Under certain conditions, we have

$$\sqrt{N}\big(\widehat{\beta} - \beta\big) \xrightarrow{D} N(0, \sigma^2)$$

  - See Wager (2024, Chapter 4.1) and Park and Kang (2022) for technical details

PLM-Learner

```
m.Lasso <- cv.glmnet(x=X, y=Y)
m.gamma <- as.vector(coef(m.Lasso, s = "lambda.1se"))
m.est <- cbind(1,X)%*%m.gamma

e.Lasso <- cv.glmnet(x=X,y=A,family="binomial")
e.gamma <- as.vector(coef(e.Lasso, s = "lambda.1se"))
e.est <- expit(cbind(1,X)%*%e.gamma)

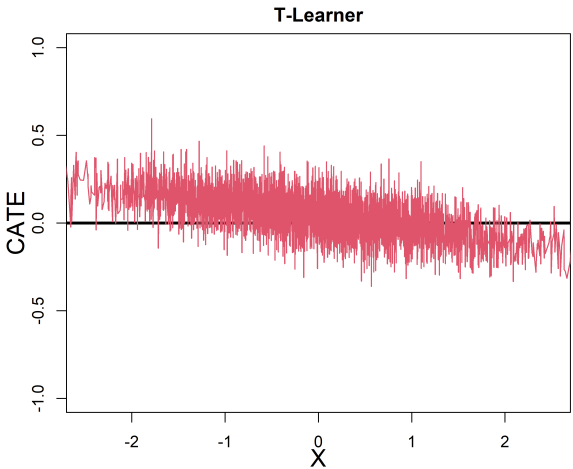
Y.res <- Y-m.est; A.res <- A-e.est; AX.res <- matrix(A.res,N,100)*X

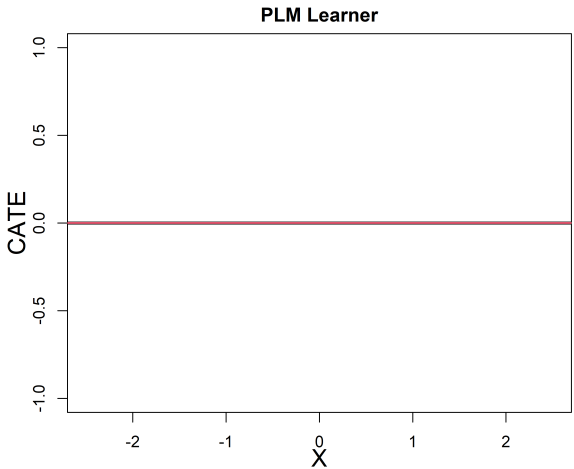
P.Lasso <- cv.glmnet(x=AX.res,y=Y.res)
P.beta <- as.vector(coef(P.Lasso, s = "lambda.1se"))

CATE.P.Est <- cbind(1,X)%*%P.beta

plot(X[,1],CATE.True,type='l',xlim=c(-2.5,2.5),ylim=c(-1,1),xlab="",ylab="",lwd=3)
par(new=TRUE)
points(X[,1],CATE.P.Est,type='l',pch=19,cex=0.25,col=2,xlab="",ylab="",lwd=2)
title(xlab="X",ylab="CATE",cex.lab=1.5,line=2)
title(main="PLM Learner")
```

PLM-Learner


T-Learner
CATE
1.0
0.5
0.0
-0.5
-1.0
-2
-1
0
1
2
X


PLM Learner
CATE
1.0
0.5
0.0
-0.5
-1.0
-2
-1
0
1
2
X

R-Learner

- The PLM-Learner can be expressed as

$$\widehat{\beta} = \arg\min_{\beta} \left[\frac{1}{N} \sum_{i=1}^{N} L\big(O_i; \beta^\intercal \psi(X)\big) + \mathcal{P}(\beta) \right] \qquad \begin{cases} L(O;\tau) = \big[\{Y - \widehat{m}(X)\} - \{A - \widehat{e}(X)\}\tau(X)\big]^2 \\ \mathcal{P} = \text{regularization term} \end{cases}$$

- More generally, one can consider the following **R-learner** (Nie and Wager, 2020):

Algorithm 5: R-Learner (R=Residualization)

Estimate $m(X)$ and $e(X)$;

Solve the following empirical loss minimization:

$$\widehat{\tau}_R = \arg\min_{\tau \in \mathcal{T}} \left[\frac{1}{N} \sum_{i=1}^{N} L\big(O_i; \tau\big) + \mathcal{P}(\tau) \right], \quad L(O;\tau) = \big[\{Y - \widehat{m}(X)\} - \{A - \widehat{e}(X)\}\tau(X)\big]^2$$

where $\mathcal{T}$ denotes a function class, possibly induced by flexible nonparametric methods (e.g., random forests, boosting);

return $\widehat{\tau}_R(X)$

- Generalized Random Forest (Athey et al., 2019): a specific instance of the R-learner framework where random forests are employed to estimate treatment effect heterogeneity
  - Implemented in `grf` R package

R-Learner

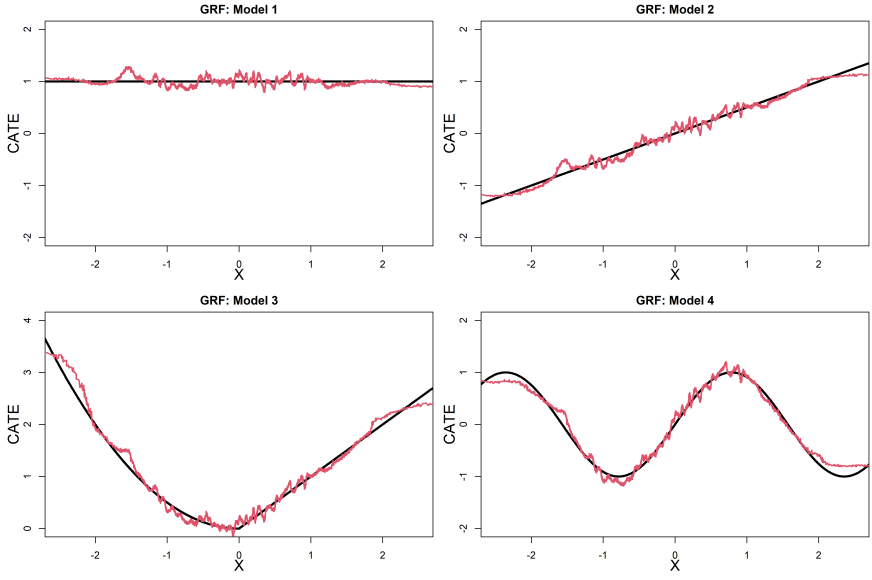
```
library(grf)

expit <- function(v){exp(v)/(1+exp(v))}
N <- 5000
X <- matrix(sort(rnorm(N)),N,1)
A <- rbinom(N,1,expit(0.5*X))
E1 <- 0.5*rnorm(N); E0 <- 0.5*rnorm(N)

mu1 <- rep(1,N); mu0 <- rep(0,N)
Y1 <- mu1 + E1; Y0 <- mu0 + E0; Y <- A*Y1+(1-A)*Y0
CATE.True.Model1 <- mu1-mu0
GRF.Model1 <- causal_forest(X=X,Y=Y,W=A)
GRF.Model1.est <- predict(GRF.Model1)

plot(X,CATE.True.Model1,type='l',xlim=c(-2.5,2.5),ylim=c(-2,2),xlab="",ylab="",lwd=3)
par(new=TRUE)
points(X,GRF.Model1.est$predictions,type='l',pch=19,cex=0.25,col=2,xlab="",ylab="",lwd=2)
title(xlab="X",ylab="CATE",cex.lab=1.5,line=2)
title(main="GRF: Model 1")
```

R-Learner



DR-Learner

- The R-Learner is motivated from the PLM formulation of the CATE

- From the DR representation of the ATE, one can show that

$$\tau_{\mathsf{CATE}}(X) = \mathrm{E}\left[\frac{A\{Y-\mu_1(X)\}}{e(X)} - \frac{(1-A)\{Y-\mu_0(X)\}}{1-e(X)} + \mu_1(X) - \mu_0(X) \,\middle|\, X\right]$$

- DR-Learner (Kennedy, 2023; Kennedy et al., 2024)

Algorithm 6: DR-Learner

Estimate $\mu(a, X)$ and $e(X)$;

Generate a pseudo-outcome:

$$\widetilde{Y} = \frac{A\{Y-\widehat{\mu}_1(X)\}}{\widehat{e}(X)} - \frac{(1-A)\{Y-\widehat{\mu}_0(X)\}}{1-\widehat{e}(X)} + \widehat{\mu}_1(X) - \widehat{\mu}_0(X)$$

Fit a model $\texttt{lm}(\widetilde{Y} \sim X)$ or $\texttt{ML}(\widetilde{Y} \sim X)$ to obtain $\widehat{\tau}_{DR}(X)$;

return $\widehat{\tau}_{DR}(X)$

DR-Learner

```
library(splines)

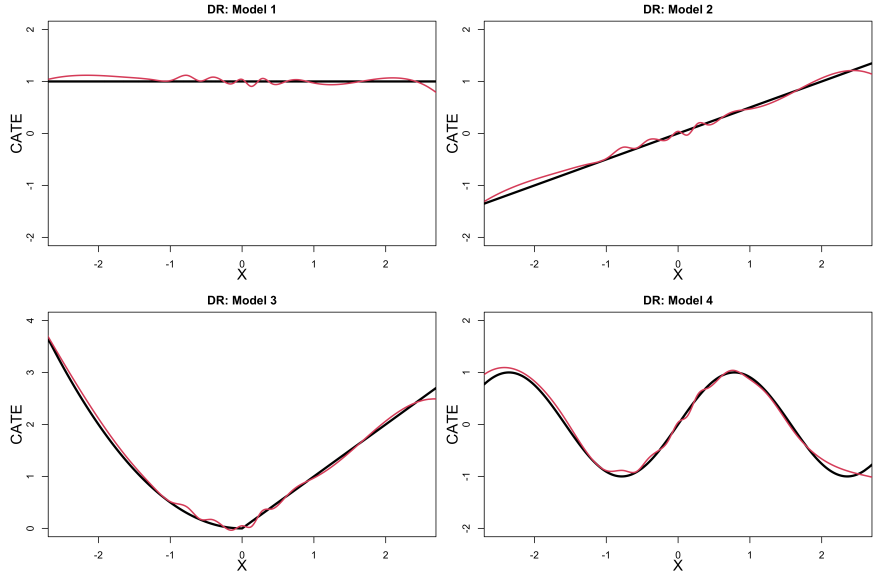
DR.CATE <- function(Y,A,X){
  ## Using Spline
  X.spline <- bs(X,df=20,intercept=FALSE)
  Data.Full <- data.frame(cbind(Y,A,X.spline))
  colnames(Data.Full) <- c("Y","A",sprintf("X%0.3d",1:ncol(X.spline)))

  PS.Fit <- glm(A~.,data=Data.Full[,-1],family="binomial")
  PS.Est <- predict(PS.Fit,type="response")

  mu1.Fit <- lm(Y~.,data=Data.Full[A==1,-2]); mu0.Fit <- lm(Y~.,data=Data.Full[A==0,-2])
  mu1.Est <- predict(mu1.Fit,Data.Full); mu0.Est <- predict(mu0.Fit,Data.Full)
  P.Y <- A*(Y-mu1.Est)/PS.Est-(1-A)*(Y-mu0.Est)/(1-PS.Est)+(mu1.Est-mu0.Est)

  Data.Full.P.Y <- data.frame(cbind(P.Y,X.spline))
  colnames(Data.Full.P.Y) <- c("PY",sprintf("X%0.3d",1:ncol(X.spline)))
  DR.CATE.Fit <- lm(PY~.,data=Data.Full.P.Y)
  return( predict(DR.CATE.Fit) )
}
```

DR-Learner



Other Approaches

- Modified covariate learner (Tian et al., 2014)
- Dragon Net (Shi et al., 2019)
- Subgroup CATE (Park and Kang, 2022)
- Forster-Warmuth Learner (Yang et al., 2024)

Minimax Rates for the CATE Estimation

Theorem (Theorems 1 and 2 of Kennedy et al. (2024))

For $x_0 \in (0,1)^d$, let $\mathcal{M}$ denote the model where

- *$P(X = x)$ is bounded above by a constant with the support $[0,1]^d$;*
- *$e(x)$ is α-Hölder smooth; $\mu(0,x)$ is β-Hölder smooth; $\tau_{CATE}(x)$ is γ-Hölder smooth*

Let $s := \frac{\alpha+\beta}{2}$. The minimax rate is lower bounded as

$$\inf_{\widehat{\tau}} \sup_{P \in \mathcal{M}} \mathrm{E}_P\left[\left|\widehat{\tau}(x_0) - \tau_{CATE}(x_0)\right|\right] \gtrsim \begin{cases} N^{-\frac{1}{1+d/(2\gamma)+d/(4s)}} & \text{if } s < \frac{d/4}{1+d/(2\gamma)} \quad = \text{low smoothness regime} \\ N^{-\frac{1}{2+d/\gamma}} = N^{-\frac{\gamma}{2\gamma+d}} & \text{if } s \geqslant \frac{d/4}{1+d/(2\gamma)} \quad = \text{high smoothness regime} \end{cases}$$

Also, one can construct a CATE estimator that attains the lower bound

Minimax Rates for the CATE Estimation

$e(x)$ is α-smooth; $\mu(0,x)$ is β-smooth; $\tau_{\mathsf{CATE}}(x)$ is γ-smooth; $s := \frac{\alpha+\beta}{2}$

$$\inf_{\widehat{\tau}} \sup_{P \in \mathcal{M}} \mathrm{E}_P\big[|\widehat{\tau}(x_0) - \tau_{\mathsf{CATE}}(x_0)|\big] \gtrsim \begin{cases} N^{-\frac{1}{1+d/(2\gamma)+d/(4s)}} & \text{if } s < \frac{d/4}{1+d/(2\gamma)} \quad = \text{low smoothness regime} \\ N^{-\frac{1}{2+d/\gamma}} = N^{-\frac{\gamma}{2\gamma+d}} & \text{if } s \geqslant \frac{d/4}{1+d/(2\gamma)} \quad = \text{high smoothness regime} \end{cases}$$

- Recall that the DR-Learner consists of two stages:
  - (Stage 1) estimation of the nuisance functions μ and e; (Stage 2) estimation of the CATE using the pseudo-outcome

- In the high smoothness regime,
  - $e(x)$ and $\mu(0,x)$ are sufficiently smooth compared to $\tau_{\mathsf{CATE}}(x)$
  - Stage 1 is therefore relatively easier than Stage 2
  - The MM rate of τ_{CATE} is dominated by Stage 2, reducing to the classic MM rate of ft estimation (Tsybakov, 2008)

- In the low smoothness regime,
  - $e(x)$ and $\mu(0,x)$ are substantially less smooth than $\tau_{\mathsf{CATE}}(x)$
  - Stage 1 is no longer ignorable, and its estimation error propagates into Stage 2
  - As a result, the MM rate of τ_{CATE} is slower than in the high smoothness regime

Outline

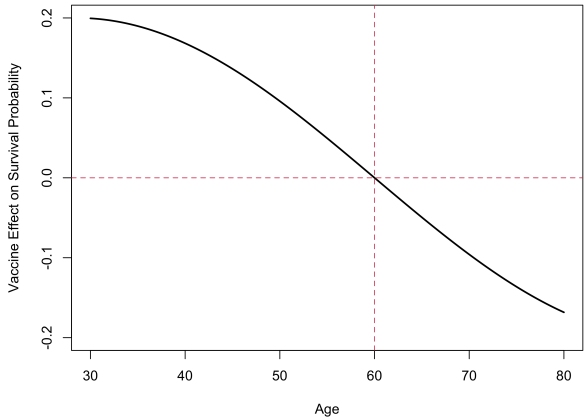
1 Heterogeneous Treatment Effect

2 (Single Time-point) Optimal Treatment Regime

3 Assignment

Example

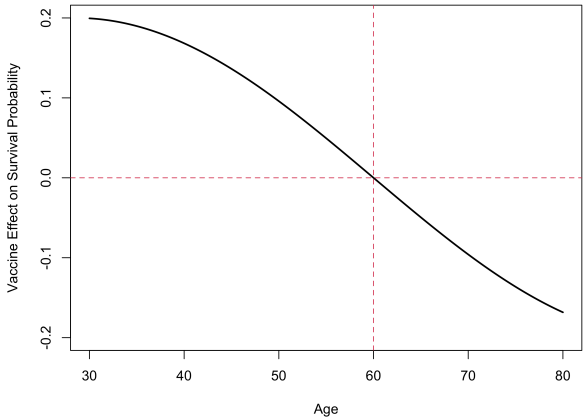
- $X =$ age, $A =$ new vaccine, $Y =$ survival
- $\tau_{\mathsf{CATE}}(x) =$ vaccine effect on survival probability given an age value
- To whom would you assign the new vaccine?



Example

- $X =$ age, $A =$ new vaccine, $Y =$ survival
- $\tau_{\mathsf{CATE}}(x) =$ vaccine effect on survival probability given an age value
- To whom would you assign the new vaccine?

Assign the new vaccine	if $X < 60$
Do not assign the new vaccine	if $X \geqslant 60$



- Optimal treatment regime (aka optimal policy learning, personalized medicine, etc): a formalization of this intuition

Framework

- Consider a policy class $\mathcal{D} = \{d \,|\, d(X) \in \{0, 1\}\}$
  - $d(X) = \mathbb{1}(X > 55)$: assign the treatment to individuals whose age X is greater than 55, and do not assign it otherwise

- $Y(d)$: potential outcome under policy $d \in \mathcal{D}$

$$Y(d) = Y(1)d(X) + Y(0)\{1 - d(X)\}$$

- $\mathcal{V}(d) = \mathrm{E}\{Y(d)\}$: value function of the policy d
  - A larger value indicates that the policy yields a higher expected outcome

- $d_{\text{opt}} \in \mathcal{D}$: optimal treatment rule; this is the rule that maximizes the expected outcome

$$d_{\text{opt}} = \underset{d \in \mathcal{D}}{\arg\max}\, \mathcal{V}(d)$$

Closed-form Representation of the OTR

- Under SUTVA:

$$d_{\text{opt}}(X) = \mathbb{1}\{\tau_{\text{CATE}}(X) \geqslant 0\}$$

- Proof

$$\begin{aligned}
d_{\text{opt}} &= \arg\max_{d \in \mathcal{D}} \mathcal{V}(d) \\
&= \arg\max_{d \in \mathcal{D}} \mathrm{E}\big\{Y(d)\big\} \\
&= \arg\max_{d \in \mathcal{D}} \mathrm{E}\big\{Y(1)d(X) + Y(0)\{1 - d(X)\}\big\} \\
&= \arg\max_{d \in \mathcal{D}} \mathrm{E}\big[\mathrm{E}\big\{Y(1)d(X) + Y(0)\{1 - d(X)\} \,\big|\, X\big\}\big] \\
&= \arg\max_{d \in \mathcal{D}} \mathrm{E}\big[d(X)\mathrm{E}\big\{Y(1) \,\big|\, X\big\} - d(X)\mathrm{E}\big\{Y(0) \,\big|\, X\big\} + \mathrm{E}\big\{Y(0) \,\big|\, X\big\}\big] \\
&= \arg\max_{d \in \mathcal{D}} \mathrm{E}\big\{d(X)\tau_{\text{CATE}}(X)\big\}
\end{aligned}$$

- To maximize $\mathrm{E}\big\{d(X), \tau_{\text{CATE}}(X)\big\}$, choose $d(X) = 1$ when $\tau_{\text{CATE}}(X) \geqslant 0$, and $d(X) = 0$ when $\tau_{\text{CATE}}(X) < 0$

Identification of the OTR

- Under SUTVA, Cond. Ign., Positivity:

$$
\begin{aligned}
d_{\text{opt}}(X) &= \mathbb{1}\{\tau_{\text{CATE}}(X) \geqslant 0\} && \text{(SUTVA)} \\
&= \mathbb{1}\{\mathrm{E}(Y|A=1,X) - \mathrm{E}(Y|A=0,X) \geqslant 0\} && \text{(Identification of the CATE)}
\end{aligned}
$$

- Note that d_{opt} is a d-dimensional function, just like τ_{CATE}, so many phenomena observed in CATE estimation also carry over to OTR estimation

Q-Learning

- Q-Learning; Indirect Approach (Watkins and Dayan, 1992; Moodie et al., 2012):

Algorithm 7: Q-Learning (Q=Quality)

Estimate $\mu(a, X)$ for $a = 0, 1$ from $\texttt{lm}(Y \sim X \mid A = a)$ or $\texttt{ML}(Y \sim X \mid A = a)$;
return $\widehat{d}_Q(X) = \mathbb{1}\{\widehat{\mu}(1, X) - \widehat{\mu}(0, X) \geqslant 0\}$

- Similar to T-Learner and S-Learner in the CATE estimation
- Pros: Simple, Easy, Intuitive
- Cons:
  - Regularization bias: $\widehat{d}_Q$ can be overly complex even when d_{opt} is simple
  - Sensitive to the specification of μ
  - $\widehat{d}_Q$ may lack a closed-form representation

Q-Learning

```
expit <- function(v){exp(v)/(1+exp(v))}
N <- 1000
X <- matrix(rnorm(N*10),N,10); X <- X[order(X[,1]),]
A <- rbinom(N,1,expit(0.5*X[,1]-1))
mu0 <- X%*%c(rep(1,5),rep(0,5)); CATE.True <- 0.25+0.5*X[,1]; mu1 <- mu0 + CATE.True
Y1 <- mu1 + rnorm(N); Y0 <- mu0 + rnorm(N); Y <- A*Y1+(1-A)*Y0
OTR.True <- as.numeric(CATE.True >= 0)

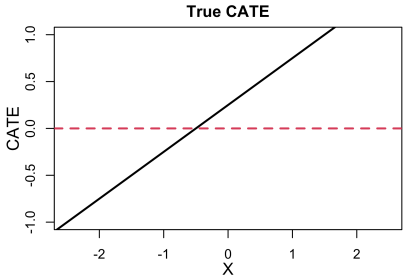
library(glmnet)

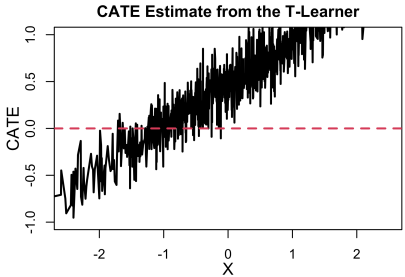
mu1.Lasso <- cv.glmnet(x=X[A==1,], y=Y[A==1]); mu0.Lasso <- cv.glmnet(x=X[A==0,], y=Y[A==0])

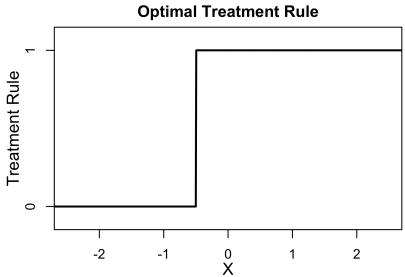
beta1 <- as.vector(coef(mu1.Lasso, s = "lambda.1se"))
beta0 <- as.vector(coef(mu0.Lasso, s = "lambda.1se"))

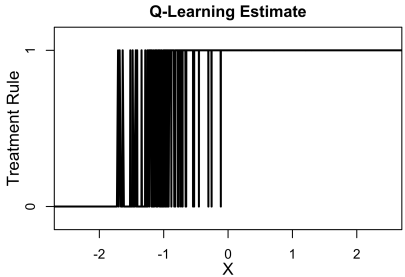
CATE.Est <- cbind(1,X)%*%(beta1-beta0); OTR.Est <- as.numeric(CATE.Est >= 0)
```

Q-Learning


True CATE
CATE
1.0
0.5
0.0
-0.5
-1.0
-2
-1
0
1
2
X


CATE Estimate from the T-Learner
CATE
1.0
0.5
0.0
-0.5
-1.0
-2
-1
0
1
2
X


Optimal Treatment Rule
Treatment Rule
1
0
-2
-1
0
1
2
X


Q-Learning Estimate
Treatment Rule
1
0
-2
-1
0
1
2
X

A-Learning

- A-Learning (Murphy, 2003; Robins, 2004; Blatt et al., 2004):

Algorithm 8: A-Learning (A=Advantage)

Estimate $\mu(0, X)$ and $e(X)$;

Estimate the *blip function* $\Delta(X) = \Delta(X; \beta)$ from the following moment equation:

$$\widehat{\beta} \quad \text{solves} \quad 0 = \frac{1}{N} \sum_{i=1}^{N} \left[\frac{\partial \Delta(X_i; \beta)}{\partial \beta} \Big\{ Y_i - \widehat{\mu}(0, X_i) - A_i \Delta(X; \beta) \Big\} \Big\{ A_i - \widehat{e}(X_i) \Big\} \right]$$

return $\widehat{d}_A(X) = \mathbb{1}\{\Delta(X; \widehat{\beta}) \geqslant 0\}$

- $\widehat{d}_A$ is robust to the specification of $\mu(0, X)$ as long as $e(X)$ is correctly specified
  - In the case of randomized experiments, this robustness property comes automatically
- In cases where $e(X)$ is mis-specified, the validity of $\widehat{d}_A$ hinges on the correct specification of $\mu(0, X)$
- In fact, the blip function$\Delta(X)$ is conceptually equal to the CATE, as $\mu(A, X) = \mu(0, X) + A\Delta(X)$
- One can also consider the R-Learner-like moment equation:

$$\widehat{\beta} \quad \text{solves} \quad 0 = \frac{1}{N} \sum_{i=1}^{N} \left[\frac{\partial \Delta(X_i; \beta)}{\partial \beta} \Big[Y_i - \widehat{\mu}(0, X_i) - \{A_i - \widehat{e}(X_i)\} \Delta(X; \beta) \Big] \Big\{ A_i - \widehat{e}(X_i) \Big\} \right]$$

A-Learning

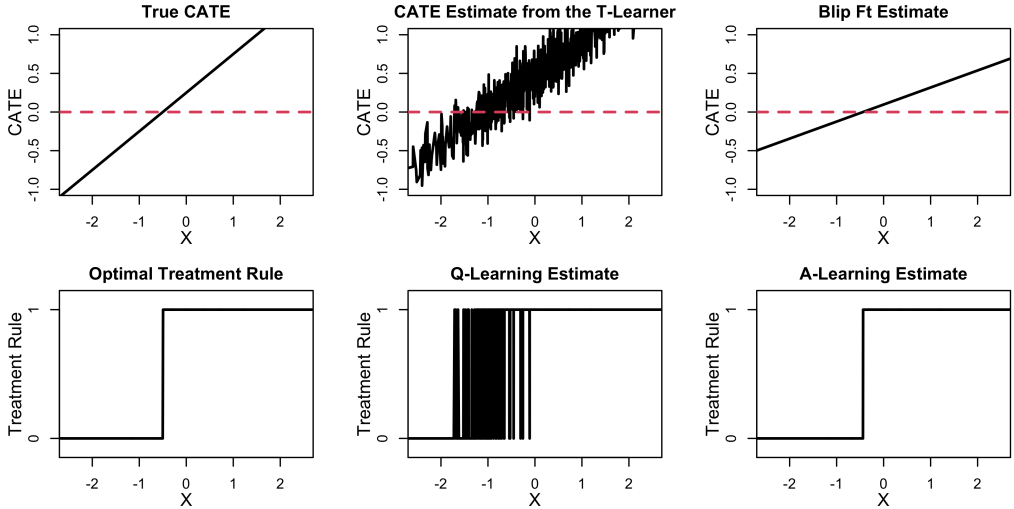
```
library(glmnet)

mu0.Lasso <- cv.glmnet(x=X[A==0,], y=Y[A==0])
beta0 <- as.vector(coef(mu0.Lasso, s = "lambda.1se"))
mu0.Est <- cbind(1,X)%*%beta0

PS.Lasso <- cv.glmnet(x=X, y=A, family="binomial")
beta.PS <- as.vector(coef(PS.Lasso, s = "lambda.1se"))
PS.Est <- expit(cbind(1,X)%*%beta.PS)

Blip.Variables <- cbind(1,X)
Blip.X.Matrix <- matrix(A-PS.Est,N,ncol(Blip.Variables))*Blip.Variables
Blip.Lasso <- cv.glmnet(x=Blip.X.Matrix, y=Y, offset=mu0.Est, intercept=FALSE )
beta.Blip <- as.vector(coef(Blip.Lasso, s = "lambda.1se"))[-1]
Blip.Est <- Blip.Variables%*%beta.Blip
OTR.A.Est <- as.numeric(Blip.Est>=0)
```

A-Learning


True CATE
CATE Estimate from the T-Learner
Blip Ft Estimate
Optimal Treatment Rule
Q-Learning Estimate
A-Learning Estimate
CATE
Treatment Rule
X
-1.0
-0.5
0.0
0.5
1.0
0
1
-2
-1
0
1
2

O-Learning

- O-Learning; Classification Approach; Direct Approach: (Zhao et al., 2012, 2015)

- The value function can be written as

$$\mathcal{V}(d) = \mathrm{E}\{Y(d)\} = \mathrm{E}\left[\frac{\mathbb{1}\{A = d(X)\}Y}{\Pr(A|X)}\right]$$

- Proof

$$\begin{aligned}
\mathrm{E}\left[\frac{\mathbb{1}\{A = d(X)\}Y}{\Pr(A|X)}\right] &= \mathrm{E}\left[\mathrm{E}\left[\frac{\mathbb{1}\{A = d(X)\}Y}{\Pr(A|X)}\,\middle|\, X\right]\right] \\
&= \mathrm{E}\left[d(X)\mathrm{E}\left\{\frac{\mathbb{1}(A = 1)Y}{\Pr(A|X)}\,\middle|\, X\right\} + \{1 - d(X)\}\mathrm{E}\left\{\frac{\mathbb{1}(A = 0)Y}{\Pr(A|X)}\,\middle|\, X\right\}\right] \\
&= \mathrm{E}\left[d(X)\mathrm{E}\left\{\frac{\mathbb{1}(A = 1)Y(1)}{\Pr(A|X)}\,\middle|\, X\right\} + \{1 - d(X)\}\mathrm{E}\left\{\frac{\mathbb{1}(A = 0)Y(0)}{\Pr(A|X)}\,\middle|\, X\right\}\right] \\
&= \mathrm{E}\big[d(X)\mathrm{E}\{Y(1)|X\} + \{1 - d(X)\}\mathrm{E}\{Y(0)|X\}\big] \\
&= \mathrm{E}\big[d(X)Y(1) + \{1 - d(X)\}Y(0)\big] \\
&= \mathrm{E}\{Y(d)\} = \mathcal{V}(d)
\end{aligned}$$

O-Learning

- In addition, we have

$$\mathcal{V}(d) = \mathrm{E}\left[\frac{\mathbb{1}\{A = d(X)\}Y}{\Pr(A|X)}\right] = \mathrm{E}\left[\frac{Y - \mathbb{1}\{A \neq d(X)\}Y}{\Pr(A|X)}\right] = C - \mathrm{E}\left[\frac{\mathbb{1}\{A \neq d(X)\}Y}{\Pr(A|X)}\right]$$

- Therefore, the OTR can be characterized as

$$d_{\text{opt}} = \arg\min_{d \in \mathcal{D}} \mathrm{E}\left[\frac{\mathbb{1}\{A \neq d(X)\}Y}{\Pr(A|X)}\right] = \arg\min_{d \in \mathcal{D}} \mathrm{E}\{L(O, d)\}$$

- d_{opt} is the optimal classifier (aka Bayes classifier) that minimizes the mis-classification risk
  - Suppose $Y \in \{0, 1\}$. Then, $L(O, d)$ is positive whenever $\mathbb{1}\{A \neq d(X)\} = 1$ and $Y = 1$;
    i.e., unit's received treatment (A) differs from the recommendation (d), but the realized outcome is favorable $(Y = 1)$
  - $L(O, d)$ is a weighted zero-one loss, i.e.,

$$L(O, d) = w(O) \times \mathbb{1}\{d \text{ is a mis-classified rule}\}\ , \quad w(O) = \frac{Y}{\Pr(A|X)}$$

O-Learning

- In addition,

$$\mathrm{E}\left[\frac{\mathbb{1}\{A \neq d(X)\}Y}{\Pr(A|X)}\right] = \mathrm{E}\left[\frac{\mathbb{1}\{(2A-1)d(X) \leqslant 0\}Y}{\Pr(A|X)}\right] = \mathrm{E}\left[\frac{\mathbb{1}\{(2A-1)\mathsf{sgn}(Y)d(X) \leqslant 0\}|Y|}{\Pr(A|X)}\right] + C$$

- Since d_{opt} is the minimizer of the risk function, we may estimate it via empirical risk minimization

$$\widehat{d}_{\mathsf{opt}} = \arg\min_{d\in\mathcal{D}} \frac{1}{N}\sum_{i=1}^{N} \frac{\mathbb{1}\{(2A_i-1)\mathsf{sgn}(Y_i)d(X_i) \leqslant 0\}|Y_i|}{\widehat{\Pr}(A_i|X_i)} = \arg\min_{f\in\mathcal{F}} \frac{1}{N}\sum_{i=1}^{N} \frac{\mathbb{1}\{(2A_i-1)\mathsf{sgn}(Y_i)f(X_i) \leqslant 0\}|Y_i|}{\widehat{\Pr}(A_i|X_i)}$$

- Note that $d(X_i) \in \{0,1\}$ can be expressed through a real-valued function $f(X_i)$ s.t. $d(X_i) = \mathbb{1}\{f(X_i) \geqslant 0\}$

- However, solving this minimization problem directly is challenging:
  1. The indicator function $\mathbb{1}\{(2A_i-1)f(X_i) \leqslant 0\}$ is non-smooth
  2. The decision space $\mathcal{D} = \{d|d(x) \in \{0,1\}\}$ (equivalently, the function space $\mathcal{F} = \{f|f(x) \in \mathbb{R}\}$) is too large

- Solutions:
  1. Replace the indicator function with a smooth surrogate loss
  2. Restrict attention to a smaller, more manageable function class for f

O-Learning: Surrogate Loss Function

- 0-1 Loss

$$L(O, f) = \mathbb{1}\{(2A-1)\text{sgn}(Y)f(X) \leqslant 0\}$$

- Hinge Loss

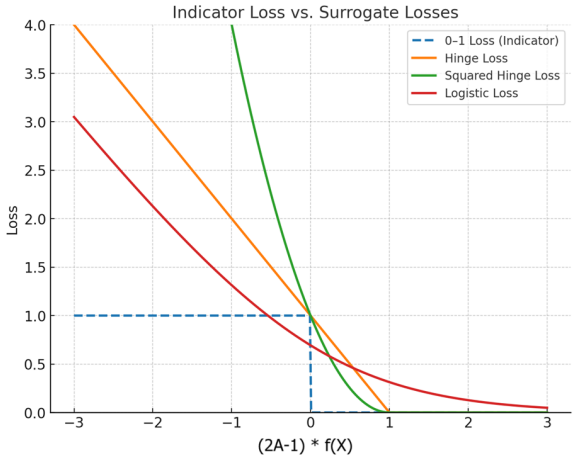
$$\begin{aligned} L_H(O, f) &= \max\{0, 1-(2A-1)\text{sgn}(Y)f(X)\} \\ &= \left\{1-(2A-1)\text{sgn}(Y)f(X)\right\}_+ \end{aligned}$$

- Squared Hinge Loss

$$L_{SH}(O, f) = L_H^2(O, f)$$

- Logistic Loss

$$L_{Log}(O, f) = \log[1+\exp\{(2A-1)\text{sgn}(Y)f(X)\}]$$



O-Learning: Model for f

- Consider simpler models for the target function f
- Parametric models:
  - $\mathcal{F} = \{f \mid f(X;\theta) = X^\intercal\theta\}$
- Approximation via basis expansions: (aka sieve)
  - $\mathcal{F} = \left\{f \mid f(X;\theta) = \sum_{k=1}^{K} \theta_k \psi_k(X)\right\}$, where ψ_k are basis functions (e.g., trigonometric functions, wavelets, splines) and K may depend on the sample size N
- Reproducing Kernel Hilbert Space (RKHS)

O-Learning: Model for f

- The infeasible optimization task:

$$\widehat{d}_{\text{opt}} = \arg\min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^{N} \frac{\mathbb{1}\{(2A_i - 1) f(X_i) \text{sgn}(Y_i) \leqslant 0\} |Y_i|}{\widehat{\Pr}(A_i | X_i)}$$

- Consider we use hinge loss as a surrogate loss and linear f

$$\mathbb{1}\{(2A-1)\text{sgn}(Y)f(X) \leqslant 0\} \quad \Rightarrow \quad \{1 - (2A-1)\text{sgn}(Y)f(X)\}_+$$

$$f(X;\theta) \quad \Rightarrow \quad X^\intercal \theta$$

- The surrogate optimization problem:

$$\widehat{d}_{\text{opt}} = \mathbb{1}(X^\intercal \widehat{\theta} \geqslant 0) \quad \text{where} \quad \widehat{\theta} = \arg\min_{\theta} \left[\frac{1}{N} \sum_{i=1}^{N} \{1 - (2A_i - 1)\text{sgn}(Y_i)(X^\intercal \theta)\}_+ \frac{|Y_i|}{\widehat{\Pr}(A_i | X_i)} + \lambda \|\theta\|_2^2 \right] \qquad (1)$$

  - The regularization term $\lambda \|f\|^2$ penalizes more complex functions, helping to prevent overfitting

- (1) is convex w.r.t. θ, so $\widehat{d}_{\text{opt}}$ can be efficiently computed

O-Learning: Quadratic Programming

- See Section 2.3 of Zhao et al. (2012) for details

1. Propose a decision function of the form $f(X;\theta) = \theta_0 + \theta_1^\intercal X$

2. Using quadratic programming algorithms find $\hat{\alpha}$ which solves the dual problem:

$$\arg\max_{\alpha} \sum_{i=1}^{N} \alpha_i - \frac{1}{2} \sum_{i=1}^{N} \sum_{j=1}^{N} \alpha_i \alpha_j (2A_i - 1)(2A_j - 1)\mathsf{sgn}(Y_i)\mathsf{sgn}(Y_j) X_i X_j$$

   subject to $0 \leqslant \alpha_i \leqslant \chi |Y_i| / \widehat{\Pr}(A_i|X_i)$ and $\sum_{i=1}^{N} \alpha_i (2A_i - 1) = 0$ where χ is a tuning parameter.

3. Obtain estimates of θ_1 via

$$\widehat{\theta}_1 = \sum_{i=1}^{N} I(\widehat{\alpha}_i > 0)\widehat{\alpha}_i (2A_i - 1)\mathsf{sgn}(Y_i) X_i$$

   and estimates of θ_0 using margin points.

4. $\widehat{d}_O(X) = \mathbb{1}\{\widehat{\theta}_0 + \widehat{\theta}_1^\intercal X \geqslant 0\}$

O-Learning

```
library(DynTxRegime)

Data <- data.frame(Y=Y,A=A,X=X)

moPropen <- buildModelObj(model = ~ X,
                          solver.method = 'glm',
                          solver.args = list('family'='binomial'),
                          predict.method = 'predict.glm',
                          predict.args = list(type='response'))

fitOWL <- owl(moPropen = moPropen,
              data = Data,
              reward = Y,
              txName="A",
              regime = ~X,
              surrogate = 'hinge',
              kernel = 'linear')

optTx(fitOWL)$optimalTx
```

O-Learning

```
library(personalized)

prop.func <- function(x, trt){
  propens.model <- cv.glmnet(y = trt, x = x, family = "binomial")
  pi.x <- predict(propens.model, s = "lambda.1se", newx = x, type = "response")[,1]
  pi.x
}

x <- as.matrix(X)
trt <- 2*A-1
y <- as.numeric(Y)

fit <- fit.subgroup(x = x, y = y, trt = trt,
                    propensity.func = prop.func,
                    loss = "owl_hinge_loss")

d_opt <- predict(fit, newx = x)
```

O-Learning

```
EmpRisk <- function(theta,Y,A,cX,PS.Est,lambda=1){
  f <- as.numeric( cX%*%theta )
  Hinge <- apply(cbind(0,(1-sign(Y)*(2*A-1)*f)),1,max)
  Weight <- as.numeric(abs(Y)*(A/PS.Est+(1-A)/(1-PS.Est)))
  mean(Hinge*Weight) + lambda*sum(theta^2)
}

O.Learning <- optim(par=Init.Para, fn=function(theta){EmpRisk(theta,Y,A,cX,PS.Est,0.01)})
OTR.Est <- as.numeric(cX%*%O.Learning$par>=0)
```

AO-Learning: Augmentation

- Augmented O-Learning (Liu et al., 2018): Let $\widehat{R}_i = Y_i - \widehat{m}(X_i)$ where $m(X) = \mathrm{E}(Y|X)$; then

$$\widehat{d}_{\text{opt,Aug}} = \mathbb{1}\{\widehat{f}(X) \geqslant 0\} \quad \text{where} \quad \widehat{f}(X) = \underset{f \in \mathcal{F}}{\arg\min} \left[\frac{1}{N} \sum_{i=1}^{N} \{1 - (2A_i - 1)|\widehat{R}_i| \widehat{f}(X)\}_+ \frac{|\widehat{R}_i|}{\widehat{\Pr}(A_i|X_i)} + \lambda \|\theta\|_2^2 \right] \tag{2}$$

- AO-L reduces stochastic variability by using residual-based weights after accounting for prognostic effects in observed outcomes

- AO-L ensures correct optimal OTRs even when the regression models used in augmentation are misspecified

O-Learning vs. AO-Learning

```
set.seed(1)
expit <- function(v){exp(v)/(1+exp(v))}
N <- 1000; X <- matrix(rnorm(N),N,1); A <- rbinom(N,1,expit(0.5*X))
E1 <- 0.5*rnorm(N); E0 <- 0.5*rnorm(N)

library(splines)
cX <- cbind(1,bs(X,df=20))

PS.Fit <- glm(A~0+cX,family="binomial")
PS.Est <- as.numeric(predict(PS.Fit,type="response"))

EmpRisk <- function(theta,Y,A,cX,PS.Est,lambda=1){
  f <- as.numeric( cX%*%theta )
  Hinge <- apply(cbind(0,(1-sign(Y)*(2*A-1)*f)),1,max)
  Weight <- as.numeric(abs(Y)*(A/PS.Est+(1-A)/(1-PS.Est)))
  mean(Hinge*Weight) + lambda*sum(theta^2)
}
```

O-Learning vs. AO-Learning

```
OTR.True <- OTR.Est <- OTR.Aug.Est <- list()
for(tt in 1:4){

  mu0 <- rep(0,N)
  if(tt==1){ mu1 <- rep(1,N) } else if (tt==2){ mu1 <- X/2
  } else if (tt==3){ mu1 <- X*as.numeric(X>0)+X^2/2*as.numeric(X<0)-1 } else if (tt==4){ mu1 <- sin(X*2) }

  Y1 <- mu1 + E1; Y0 <- mu0 + E0; Y <- A*Y1+(1-A)*Y0
  CATE.True <- mu1-mu0; OTR.True[[tt]] <- as.numeric(CATE.True>=0)
  mu1.Fit <- lm(Y[A==1]~0+cX[A==1,]); mu0.Fit <- lm(Y[A==0]~0+cX[A==0,]); m.Fit <- lm(Y~0+cX)
  Init.Para <- mu1.Fit$coefficients-mu0.Fit$coefficients
  Res <- Y - cX%*%m.Fit$coefficients

  O.Learning <- optim(par=Init.Para, fn=function(theta){EmpRisk(theta,Y,A,cX,PS.Est,0.01)})
  OTR.Est[[tt]] <- as.numeric(cX%*%O.Learning$par>=0)

  OA.Learning <- optim(par=Init.Para, fn=function(theta){EmpRisk(theta,Res,A,cX,PS.Est,0.01)})
  OTR.Aug.Est[[tt]] <- as.numeric(cX%*%OA.Learning$par>=0)
}
```

O-Learning vs. AO-Learning

```
OTR.True <- OTR.Est <- OTR.Aug.Est <- list()
for(tt in 1:4){

  mu0 <- rep(0,N)
  if(tt==1){ mu1 <- rep(1,N) } else if (tt==2){ mu1 <- X/2
  } else if (tt==3){ mu1 <- X*as.numeric(X>0)+X^2/2*as.numeric(X<0)-1 } else if (tt==4){ mu1 <- sin(X*2) }

  Y1 <- mu1 + E1; Y0 <- mu0 + E0; Y <- A*Y1+(1-A)*Y0
  CATE.True <- mu1-mu0; OTR.True[[tt]] <- as.numeric(CATE.True>=0)
  mu1.Fit <- lm(Y[A==1]~0+cX[A==1,]); mu0.Fit <- lm(Y[A==0]~0+cX[A==0,]); m.Fit <- lm(Y~0+cX)
  Init.Para <- mu1.Fit$coefficients-mu0.Fit$coefficients
  Res <- Y - cX%*%m.Fit$coefficients

  Initial.Learning <- optim(par=Init.Para, fn=function(theta){EmpRisk(theta,Y,A,cX,PS.Est,0.01)})

  O.Learning <- optim(par=Initial.Learning$par, fn=function(theta){EmpRisk(theta,Y,A,cX,PS.Est,0.01)})
  OTR.Est[[tt]] <- as.numeric(cX%*%O.Learning$par>=0)

  OA.Learning <- optim(par=Initial.Learning$par, fn=function(theta){EmpRisk(theta,Res,A,cX,PS.Est,0.01)})
  OTR.Aug.Est[[tt]] <- as.numeric(cX%*%OA.Learning$par>=0)
}
```

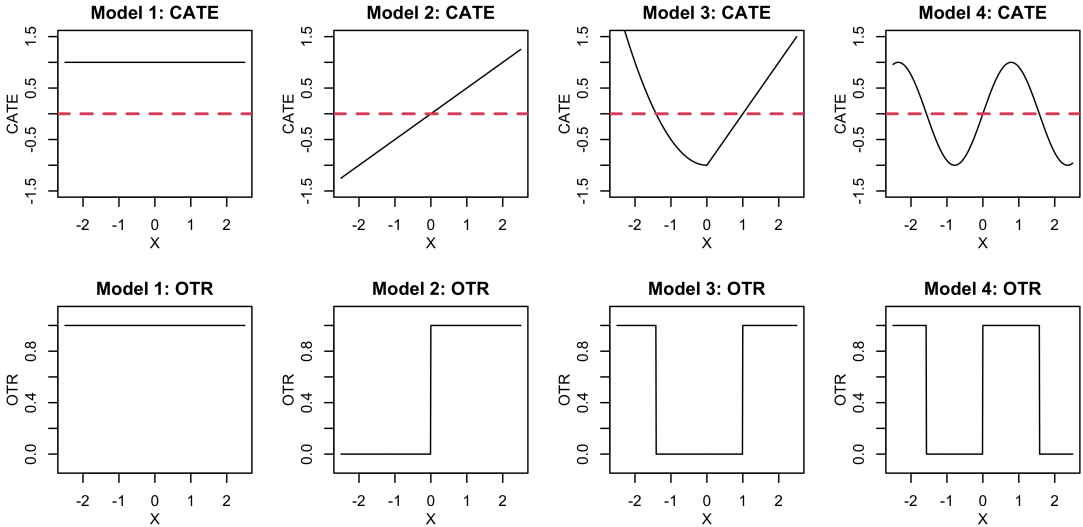
O-Learning vs. AO-Learning

```
TB <- function(v1,v2){
  table( factor(v1, levels = c(0, 1)), factor(v2, levels = c(0, 1)))
}
Summary <- function(table){
  TP <- table[2,2]; TN <- table[1,1]; FP <- table[1,2]; FN <- table[2,1]
  acc <- (TP+TN)/sum(table)
  f1 <- 2*TP/(2*TP+FP+FN)
  sensitivity <- TP/(TP+FN)
  precision <- TP/(TP+FP)
  return(c(acc,f1,sensitivity,precision))
}

TB(OTR.True[[1]],OTR.Est[[1]]); TB(OTR.True[[1]],OTR.Aug.Est[[1]])
TB(OTR.True[[2]],OTR.Est[[2]]); TB(OTR.True[[2]],OTR.Aug.Est[[2]])
TB(OTR.True[[3]],OTR.Est[[3]]); TB(OTR.True[[3]],OTR.Aug.Est[[3]])
TB(OTR.True[[4]],OTR.Est[[4]]); TB(OTR.True[[4]],OTR.Aug.Est[[4]])

Summary(TB(OTR.True[[1]],OTR.Est[[1]])); Summary(TB(OTR.True[[1]],OTR.Aug.Est[[1]]))
Summary(TB(OTR.True[[2]],OTR.Est[[2]])); Summary(TB(OTR.True[[2]],OTR.Aug.Est[[2]]))
Summary(TB(OTR.True[[3]],OTR.Est[[3]])); Summary(TB(OTR.True[[3]],OTR.Aug.Est[[3]]))
Summary(TB(OTR.True[[4]],OTR.Est[[4]])); Summary(TB(OTR.True[[4]],OTR.Aug.Est[[4]]))
```

O-Learning vs. AO-Learning



O-Learning vs. AO-Learning

Model		1				2				3				4			
Method		O-L		AO-L		O-L		AO-L		O-L		AO-L		O-L		AO-L	
Estimated OTR		0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
True OTR	0	0	0	0	0	449	69	511	7	755	0	748	7	451	72	449	74
	1	4	996	0	1000	0	482	15	467	25	220	29	216	46	431	20	457

Classification Performance Measure	Accuracy	0.996	1.000	0.931	0.978	0.975	0.964	0.882	0.906
	F1	0.998	1.000	0.933	0.977	0.946	0.923	0.880	0.907
	Sensitivity	0.996	1.000	1.000	0.969	0.898	0.882	0.904	0.958
	Precision	1.000	1.000	0.875	0.985	1.000	0.969	0.857	0.861

Excess Risk Analysis of O-Learning

- Excess risk of an OTR estimator $\widehat{d}$:

$$\mathcal{V}(d_{\text{opt}}) - \mathcal{V}(\widehat{d}) \tag{3}$$

- Since $\mathcal{V}(d_{\text{OPT}}) = \sup_{d \in \mathcal{D}} \mathcal{V}(d)$, an estimator $\widehat{d}$ is better if the gap in (3) is smaller

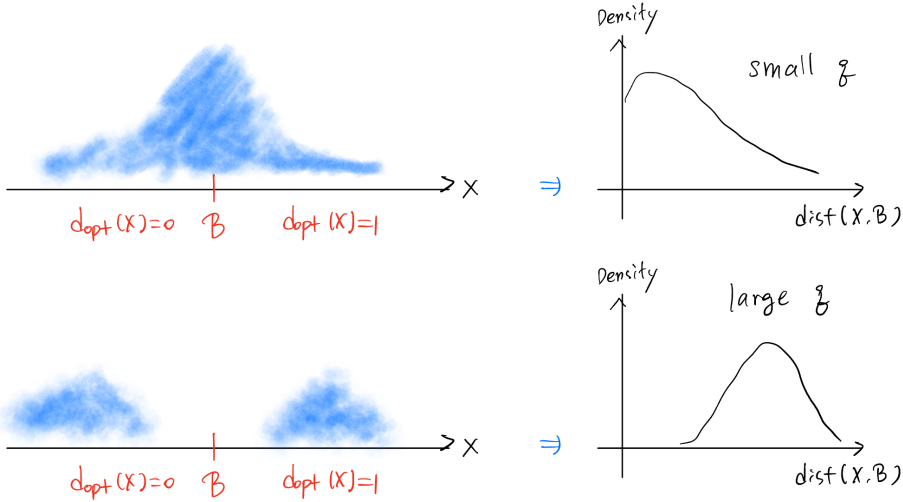
- Note that $\mathcal{V}(d_{\text{OPT}})$ is a fixed quantity, while $\mathcal{V}(\widehat{d})$ is random because it depends on $\widehat{d}$

- Statistical questions:
  - Does $\mathcal{V}(\widehat{d}) \to \mathcal{V}(d_{\text{opt}})$ as $N \to \infty$?
  - If so, at what rate?

- Difficulties:
  - Use of a surrogate loss function $\Rightarrow$ must account for surrogate error
  - True OTR may lie outside the posited space ($\mathcal{F}$ = RKHS) $\Rightarrow$ must characterize approximation error
  - Estimator differs from regression $\Rightarrow$ must rely on empirical risk minimization theory

Excess Risk Analysis of O-Learning

- Let $q \in (0, \infty)$ be a parameter controlling the dist of distance$(X, \mathcal{B})$; $\mathcal{B} = \{X \mid \tau_{\mathsf{CATE}}(X) = 0\}$ = decision boundary
  - Large q: d_{opt} is easier to learn; Small q: d_{opt} is harder to learn



Excess Risk Analysis of O-Learning

Theorem (Zhao et al. (2012))

Then, we have

$$\mathcal{V}(d_{opt}) - \mathcal{V}(\widehat{d}) = O_P\left(N^{-\frac{2q}{(4+v)q+2+(2-v)(1+\delta)}}\right), \quad \forall \delta > 0, \quad \forall v \in (0,2)$$

- If $q \simeq \infty$ $\Rightarrow$ rate $\simeq O_P(N^{-1/2})$
- If X has a continuous density near $\mathcal{B}$, we have $q = 2/\dim(X)$ $\Rightarrow$ rate $\simeq O_P(N^{-2/\{\dim(X)+2\}})$
- When X is completely separated from the decision boundary, one can improve the rate as

$$\mathcal{V}(d_{\text{opt}}) - \mathcal{V}(\widehat{d}) = O_P\left(N^{-\frac{1}{v+1}\frac{q}{q+1}}\right), \qquad \forall v \in (0,2)$$

$$\simeq O_P(N^{-1}) \qquad \text{by taking } v \simeq 0 \text{ and } q \simeq \infty$$

Optimal Treatment Regime in Other Disciplines

- Computer Science/Machine Learning:
  reinforcement learning for multi-stage decision-making and online data settings (Sutton et al., 1998)

- Economics:
  optimal policy learning, welfare maximization (Kitagawa and Tetenov, 2018; Athey and Wager, 2021)

- Biostatistics/Medicine:
  personalized medicine (Hamburg and Collins, 2010)

- Psychology/Behavioral Science:
  adaptive interventions, just-in-time adaptive interventions (JITAI) (Nahum-Shani et al., 2016)

Outline

1 Heterogeneous Treatment Effect

2 (Single Time-point) Optimal Treatment Regime

3 Assignment

Assignment

- Analyze the dataset `causalweight::JC` (Job Corps)

  While the dataset contains additional variables, we will focus only on the following:

  - $X =$ (`female, age, white, black, hispanic`)
  - $A =$ `assignment`
  - $Y =$ `earny4`

  We aim to study the effect of JC program assignment on weekly earnings in the fourth year after assignment

- Using the dataset, estimate:
  - the CATE using T-learner, GRF, and DR-learner
  - the OTR using Q-learning, O-learning, and AO-learning

- For nuisance function estimation, you may use parametric regression models or machine learning methods
  - But note that ML may take longer due to data size

- Be sure to interpret results, as understanding how to draw the causal conclusion is the key focus of this course

References

Athey, S., Tibshirani, J., and Wager, S. (2019). Generalized random forests. *The Annals of Statistics*, 47(2):1148–1178.

Athey, S. and Wager, S. (2021). Policy learning with observational data. *Econometrica*, 89(1):133–161.

Blatt, D., Murphy, S. A., and Zhu, J. (2004). A-learning for approximate planning. *Ann Arbor*, 1001:48109–2122.

Deines, J. M., Guan, K., Lopez, B., Zhou, Q., White, C. S., Wang, S., and Lobell, D. B. (2023). Recent cover crop adoption is associated with small maize and soybean yield losses in the united states. *Global change biology*, 29(3):794–807.

Hamburg, M. A. and Collins, F. S. (2010). The path to personalized medicine. *New England Journal of Medicine*, 363(4):301–304.

Kennedy, E. H. (2023). Towards optimal doubly robust estimation of heterogeneous causal effects. *Electronic Journal of Statistics*, 17(2):3008–3049.

Kennedy, E. H., Balakrishnan, S., Robins, J. M., and Wasserman, L. (2024). Minimax rates for heterogeneous causal effect estimation. *Annals of statistics*, 52(2):793.

Kitagawa, T. and Tetenov, A. (2018). Who should be treated? empirical welfare maximization methods for treatment choice. *Econometrica*, 86(2):591–616.

Kosorok, M. R. and Moodie, E. E. (2015). *Adaptive Treatment Strategies in Practice: Planning Trials and Analyzing Data for Personalized Medicine*. SIAM.

Künzel, S. R., Sekhon, J. S., Bickel, P. J., and Yu, B. (2019). Metalearners for estimating heterogeneous treatment effects using machine learning. *Proceedings of the National Academy of Sciences*, 116(10):4156–4165.

Liu, Y., Wang, Y., Kosorok, M. R., Zhao, Y., and Zeng, D. (2018). Augmented outcome-weighted learning for estimating optimal dynamic treatment regimens. *Statistics in medicine*, 37(26):3776–3788.

Moodie, E. E., Chakraborty, B., and Kramer, M. S. (2012). Q-learning for estimating optimal dynamic treatment rules from observational data. *Canadian Journal of Statistics*, 40(4):629–645.

Murphy, S. A. (2003). Optimal dynamic treatment regimes. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 65(2):331–355.

Nahum-Shani, I., Smith, S. N., Spring, B. J., Collins, L. M., Witkiewitz, K., Tewari, A., and Murphy, S. A. (2016). Just-in-time adaptive interventions (jitais) in mobile health: key components and design principles for ongoing health behavior support. *Annals of behavioral medicine*, pages 1–17.

References

Nie, X. and Wager, S. (2020). Quasi-oracle estimation of heterogeneous treatment effects. *Biometrika*, 108(2):299–319.

Park, C. and Kang, H. (2022). A groupwise approach for inferring heterogeneous treatment effects in causal inference. *Preprint arXiv: 1908.04427*. Department of Statistics, University of Wisconsin-Madison, Madison.

Robins, J. M. (2004). Optimal structural nested models for optimal sequential decisions. In *Proceedings of the Second Seattle Symposium in Biostatistics: analysis of correlated data*, pages 189–326. Springer.

Robinson, P. M. (1988). Root-n-consistent semiparametric regression. *Econometrica*, 56(4):931–954.

Rubin, D. B. (1980). Randomization analysis of experimental data: The fisher randomization test comment. *Journal of the American statistical association*, 75(371):591–593.

Shi, C., Blei, D., and Veitch, V. (2019). Adapting neural networks for the estimation of treatment effects. In Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., and Garnett, R., editors, *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc.

Sutton, R. S., Barto, A. G., et al. (1998). *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge.

Tian, L., Alizadeh, A. A., Gentles, A. J., and Tibshirani, R. (2014). A simple method for estimating interactions between a treatment and a large number of covariates. *Journal of the American Statistical Association*, 109(508):1517–1532.

Tsybakov, A. B. (2008). *Introduction to Nonparametric Estimation*. Springer Publishing Company, Incorporated, 1st edition.

Wager, S. (2024). Causal inference: A statistical learning approach.

Watkins, C. J. and Dayan, P. (1992). Q-learning. *Machine learning*, 8(3):279–292.

Yang, Y., Kuchibhotla, A. K., and Tchetgen Tchetgen, E. (2024). Forster-warmuth counterfactual regression: A unified learning approach. *arXiv Preprint:2307.16798*.

Zhao, Y., Zeng, D., Rush, A. J., and Kosorok, M. R. (2012). Estimating individualized treatment rules using outcome weighted learning. *Journal of the American Statistical Association*, 107(499):1106–1118.

Zhao, Y.-Q., Zeng, D., Laber, E. B., and Kosorok, M. R. (2015). New statistical learning methods for estimating optimal dynamic treatment regimes. *Journal of the American Statistical Association*, 110(510):583–598.